CSA 1737 – ARTIFICIAL INTELLIGENCE (Prolog)

Laboratory Record – Experiments 31 to 35

HOW TO RUN THE PROLOG PROGRAMS IN GOOGLE COLAB

Google Colab does not have SWI-Prolog by default. Install it once, then write each program into a file with %%writefile and run it with swipl. Each program ends with ":- initialization(main)." so the queries run automatically.

Cell 1 (run once):

```
!apt-get install -y -qq swi-prolog > /dev/null
```

Cell 2 (for every experiment; the first line must be %%writefile):

```
%%writefile ex26.pl
<paste the Prolog program here>
```

Cell 3 (run the program):

```
!swipl -q -s ex26.pl -t halt
```

Ex. No. 31: Birds - Can a Bird Fly or Not

AIM

To write a Prolog program to find whether a particular bird can fly or not, and to incorporate the required queries.

ALGORITHM

Step 1: Start the program.

Step 2: Create facts for all birds with bird(Name).

Step 3: Create facts for the exceptions with cannot_fly(Name), such as penguin and ostrich.

Step 4: Define the rule can_fly(X) :- bird(X), \+ cannot_fly(X).

Step 5: Run queries for particular birds and for a non-bird.

Step 6: Stop the program.

CODE

```
% Experiment 31: Can a particular bird fly or not?
bird(sparrow).
bird(eagle).
bird(parrot).
bird(crow).
bird(penguin).
bird(ostrich).
bird(kiwi).

% Birds that cannot fly (exceptions)
cannot_fly(penguin).
cannot_fly(ostrich).
cannot_fly(kiwi).

can_fly(X) :- bird(X), \+ cannot_fly(X).

% Helper to print the answer of a yes/no query
ask(Goal) :-
    ( call(Goal) -> R = true ; R = false ),
    format("?- ~w.~n~w~n~n", [Goal, R]).

main :-
    ask(can_fly(sparrow)),
    ask(can_fly(eagle)),
    ask(can_fly(penguin)),
    ask(can_fly(ostrich)),
    ask(can_fly(dog)),            % dog is not a bird
    format("?- can_fly(X).~n"),
    findall(X, can_fly(X), L1), format("X = ~w~n~n", [L1]),
    format("?- bird(X), cannot_fly(X).~n"),
    findall(X, (bird(X), cannot_fly(X)), L2), format("X = ~w~n", [L2]).

:- initialization(main).
```

Colab run command:

```
!swipl -q -s ex31.pl -t halt
```

OUTPUT

```
?- can_fly(sparrow).
true

?- can_fly(eagle).
true

?- can_fly(penguin).
false

?- can_fly(ostrich).
false

?- can_fly(dog).
false

?- can_fly(X).
X = [sparrow,eagle,parrot,crow]

?- bird(X), cannot_fly(X).
X = [penguin,ostrich,kiwi]
```

RESULT

Thus the Prolog program to find whether a bird can fly or not was written and executed successfully, and the queries gave the expected answers.

Ex. No. 32: Family Tree

AIM

To write a Prolog program to implement a family tree and define the relations mother, father, grandfather, grandmother, sister and brother.

ALGORITHM

Step 1: Start the program.

Step 2: Declare Pam, Liz, Ann and Pat as female, and Tom, Bob and Jim as male.

Step 3: Create parent(Parent, Child) facts.

Step 4: Define mother(X,Y) as parent(X,Y) and female(X); father(X,Y) as parent(X,Y) and male(X).

Step 5: Define grandfather and grandmother through father/mother and parent.

Step 6: Define sister(X,Y) and brother(X,Y): X and Y share a parent, X is female/male, and X is not Y.

Step 7: Run queries for each relation.

Step 8: Stop the program.

CODE

```
% Experiment 32: Family Tree
% Pam, Liz, Ann and Pat are female; Tom, Bob and Jim are male
female(pam).
female(liz).
female(ann).
female(pat).
male(tom).
male(bob).
male(jim).

% parent(Parent, Child)
parent(pam, bob).
parent(tom, bob).
parent(tom, liz).
parent(bob, ann).
parent(bob, pat).
parent(pat, jim).

mother(X, Y)      :- parent(X, Y), female(X).
father(X, Y)      :- parent(X, Y), male(X).
grandfather(X, Z) :- father(X, Y), parent(Y, Z).
grandmother(X, Z) :- mother(X, Y), parent(Y, Z).
sister(X, Y)      :- parent(Z, X), parent(Z, Y), female(X), X \== Y.
brother(X, Y)     :- parent(Z, X), parent(Z, Y), male(X), X \== Y.

show(Query, Template, Goal) :-
    ( setof(Template, Goal, L) -> true ; L = [] ),
    format("?- ~w.~n~w~n~n", [Query, L]).

main :-
    show('mother(X, Y)',      X-Y, mother(X, Y)),
    show('father(X, Y)',      X-Y, father(X, Y)),
```

```
    show('grandfather(X, Y)', X-Y, grandfather(X, Y)),
    show('grandmother(X, Y)', X-Y, grandmother(X, Y)),
    show('sister(X, Y)',      X-Y, sister(X, Y)),
    show('brother(X, Y)',     X-Y, brother(X, Y)),
    show('mother(X, bob)',    X,   mother(X, bob)),
    show('grandfather(tom, X)', X, grandfather(tom, X)).

:- initialization(main).
```

Colab run command:

```
!swipl -q -s ex32.pl -t halt
```

OUTPUT

```
?- mother(X, Y).
[pam-bob,pat-jim]

?- father(X, Y).
[bob-ann,bob-pat,tom-bob,tom-liz]

?- grandfather(X, Y).
[bob-jim,tom-ann,tom-pat]

?- grandmother(X, Y).
[pam-ann,pam-pat]

?- sister(X, Y).
[ann-pat,liz-bob,pat-ann]

?- brother(X, Y).
[bob-liz]

?- mother(X, bob).
[pam]

?- grandfather(tom, X).
[ann,pat]
```

RESULT

Thus the Prolog program for the family tree was written and executed successfully, and all the relations were verified.

Ex. No. 33: Dieting System Based on Disease

AIM

To write a Prolog program to suggest a dieting system based on the disease.

ALGORITHM

Step 1: Start the program.

Step 2: Create facts diet(Disease, Diet) and avoid(Disease, Food) for various diseases.

Step 3: Create facts patient(Name, Disease).

Step 4: Define the rule suggest(Patient, Diet) that finds the disease of the patient and its diet.

Step 5: Run queries to get the diet for a disease and for a patient.

Step 6: Stop the program.

CODE

```
% Experiment 33: Dieting system based on disease
diet(diabetes,     'Low sugar, high fibre foods, whole grains, vegetables').
diet(hypertension, 'Low salt, fruits, vegetables, avoid pickles and processed
food').
diet(obesity,      'Low calorie, high protein, plenty of vegetables, avoid fried
food').
diet(anaemia,      'Iron rich foods like spinach, dates, beetroot, and vitamin C
fruits').
diet(gastritis,    'Bland, low spice, small frequent meals, avoid tea and
coffee').
diet(typhoid,      'Soft, easily digestible, low fibre food with plenty of
fluids').

avoid(diabetes,     sweets).
avoid(hypertension, salt).
avoid(obesity,      fried_food).
avoid(anaemia,      tea_after_meals).
avoid(gastritis,    spicy_food).

patient(ravi,  diabetes).
patient(priya, anaemia).
patient(arun,  hypertension).
patient(meena, gastritis).

suggest(Patient, Diet) :- patient(Patient, Disease), diet(Disease, Diet).

main :-
    format("?- diet(diabetes, D).~n"),
    diet(diabetes, D1), format("D = ~w~n~n", [D1]),

    format("?- avoid(hypertension, F).~n"),
    avoid(hypertension, F), format("F = ~w~n~n", [F]),

    format("?- suggest(priya, D).~n"),
    suggest(priya, D2), format("D = ~w~n~n", [D2]),

    format("?- suggest(Patient, Diet).  (all patients)~n"),
    forall(suggest(P, D3), format("~w -> ~w~n", [P, D3])).
```

```
:- initialization(main).
```

Colab run command:

```
!swipl -q -s ex33.pl -t halt
```

OUTPUT

```
?- diet(diabetes, D).
D = Low sugar, high fibre foods, whole grains, vegetables

?- avoid(hypertension, F).
F = salt

?- suggest(priya, D).
D = Iron rich foods like spinach, dates, beetroot, and vitamin C fruits

?- suggest(Patient, Diet).  (all patients)
ravi -> Low sugar, high fibre foods, whole grains, vegetables
priya -> Iron rich foods like spinach, dates, beetroot, and vitamin C fruits
arun -> Low salt, fruits, vegetables, avoid pickles and processed food
meena -> Bland, low spice, small frequent meals, avoid tea and coffee
```

RESULT

Thus the Prolog program to suggest a diet based on the disease was written and executed successfully, and the output was verified.

Ex. No. 34: Monkey and Banana Problem

AIM

To write a Prolog program to implement the Monkey and Banana problem.

ALGORITHM

Step 1: Start the program.

Step 2: Represent a state as state(MonkeyPosition, MonkeyOnBoxOrFloor, BoxPosition, HasBanana).

Step 3: Define the moves: walk, push the box, climb the box and grasp the banana.

Step 4: The banana hangs from the ceiling in the middle of the room, so the monkey can grasp it only when it is on the box in the middle.

Step 5: Define canget(State, Plan), which finds a sequence of moves that leads to a state where the monkey has the banana.

Step 6: Query canget from the initial state to obtain the plan.

Step 7: Stop the program.

CODE

```
% Experiment 34: Monkey and Banana problem
% state(MonkeyHorizontal, MonkeyVertical, BoxPosition, HasBanana)
% The banana hangs from the ceiling in the middle of the room.

move(state(middle, onbox, middle, hasnot), grasp,        state(middle, onbox,
middle, has)).
move(state(P, onfloor, P, H),              climb,        state(P, onbox, P, H)).
move(state(P1, onfloor, P1, H),            push(P1, P2), state(P2, onfloor, P2,
H)).
move(state(P1, onfloor, B, H),             walk(P1, P2), state(P2, onfloor, B,
H)).

% canget(State, Plan)
canget(state(_, _, _, has), []).
canget(State1, [Move|Moves]) :-
    move(State1, Move, State2),
    canget(State2, Moves).

main :-
    Start = state(atdoor, onfloor, atwindow, hasnot),
    format("Initial state: ~w~n~n", [Start]),
    format("?- canget(~w, Plan).~n", [Start]),
    canget(Start, Plan),
    format("Plan found:~n"),
    forall(member(Step, Plan), format("   ~w~n", [Step])),
    format("~n?- canget(~w, _).~ntrue~n", [Start]).

:- initialization(main).
```

Colab run command:

```
!swipl -q -s ex34.pl -t halt
```

OUTPUT

```
Initial state: state(atdoor,onfloor,atwindow,hasnot)

?- canget(state(atdoor,onfloor,atwindow,hasnot), Plan).
Plan found:
   walk(atdoor,atwindow)
   push(atwindow,middle)
   climb
   grasp

?- canget(state(atdoor,onfloor,atwindow,hasnot), _).
true
```

RESULT

Thus the Prolog program for the Monkey and Banana problem was written and executed successfully, and the monkey obtained the banana in 4 moves.

Ex. No. 35: Fruit and its Colour using Backtracking

AIM

To write a Prolog program for fruits and their colours using backtracking.

ALGORITHM

Step 1: Start the program.

Step 2: Create facts fruit_color(Fruit, Color).

Step 3: Write a rule that calls fruit_color(Fruit, Color), prints the pair and then forces failure using fail.

Step 4: Because of the failure, Prolog backtracks and finds the next matching fact until all facts have been printed.

Step 5: Run queries to list all fruits of a given colour and all colours of a given fruit.

Step 6: Stop the program.

CODE

```
% Experiment 35: Fruit and its colour using Backtracking
fruit_color(apple,      red).
fruit_color(apple,      green).
fruit_color(banana,     yellow).
fruit_color(mango,      yellow).
fruit_color(orange,     orange).
fruit_color(grapes,     green).
fruit_color(grapes,     purple).
fruit_color(strawberry, red).
fruit_color(cherry,     red).
fruit_color(lemon,      yellow).

% Backtracking: fail forces Prolog to go back and find the next solution
show_all :-
    fruit_color(Fruit, Color),
    format("~w is ~w~n", [Fruit, Color]),
    fail.
show_all.

show_color(Color) :-
    format("Fruits that are ~w:~n", [Color]),
    fruit_color(Fruit, Color),
    format("   ~w~n", [Fruit]),
    fail.
show_color(_).

main :-
    format("?- fruit_color(Fruit, Color).  (all solutions by backtracking)~n"),
    show_all,
    nl,
    format("?- fruit_color(Fruit, red).~n"),
    show_color(red),
    nl,
    format("?- fruit_color(grapes, Color).~n"),
    findall(C, fruit_color(grapes, C), L),
    format("Color = ~w~n", [L]).
```

```
:- initialization(main).
```

Colab run command:

```
!swipl -q -s ex35.pl -t halt
```

OUTPUT

```
?- fruit_color(Fruit, Color).  (all solutions by backtracking)
apple is red
apple is green
banana is yellow
mango is yellow
orange is orange
grapes is green
grapes is purple
strawberry is red
cherry is red
lemon is yellow

?- fruit_color(Fruit, red).
Fruits that are red:
   apple
   strawberry
   cherry

?- fruit_color(grapes, Color).
Color = [green,purple]
```

RESULT

Thus the Prolog program for fruits and their colours using backtracking was written and executed successfully, and the output was verified.